

SE446: Big Data Engineering

Week 7C: Lab 07 — MapReduce vs Spark

Experiment Results & Analysis

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 Lab Setup & Cluster
- 2 MapReduce Benchmark Results

Outline

1 Lab Setup & Cluster

2 MapReduce Benchmark Results

Learning Objectives

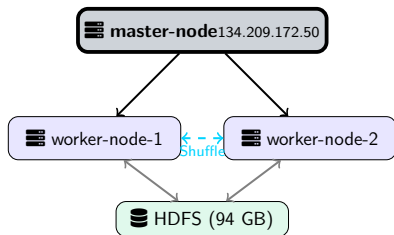
By the end of this session you will be able to:

- 1 Explain the experimental methodology behind the MapReduce vs Spark benchmark
- 2 Interpret real timing data collected from a live 3-node Hadoop/Spark cluster
- 3 Quantify Spark's caching advantage using concrete speedup ratios
- 4 Predict performance scaling for iterative workloads beyond 3 passes
- 5 Connect the measured numbers back to the architectural differences studied in W07A

Experiment

Word Count benchmark — **3 MapReduce jobs** vs **3 Spark actions** on the same 50 MB corpus, run on a live cluster with 2 YARN worker nodes.

The Cluster: Our Real Hadoop/Spark Environment



Stack:

- Hadoop **3.4.1** + YARN
- Spark **3.5.4**
- Python **3.10** (PySpark driver)
- Ubuntu 22.04

Dataset:

- 50 MB corpus, 631,664 lines
- 456 distinct words
- Replication factor: 2
- HDFS path:
`/user/akoubaa/lab07/corpus.txt`

Real Cluster

All times are from a **live run** on March 3, 2026 — no simulation.

Experimental Design

MapReduce Benchmark

- 1 **Job 1:** Full word count — count all unique words
- 2 **Job 2:** Filter words with count ≥ 100
- 3 **Job 3:** Extract top-10 most frequent words

Key Property

Each job reads the **full dataset from HDFS** independently. No data sharing between jobs.

Spark Benchmark

- 1 **Action 1:** `count()` — total distinct words
⇒ triggers HDFS read + `.cache()`
- 2 **Action 2:** Filter count ≥ 100
⇒ from executor RAM
- 3 **Action 3:** Top-10 words
⇒ from executor RAM

Key Property

HDFS is read **exactly once**. Actions 2 and 3 operate on the **cached in-memory RDD**.

Same Analysis — Different Architecture

Both frameworks compute identical results on identical data. Only the execution strategy differs.

Outline

1 Lab Setup & Cluster

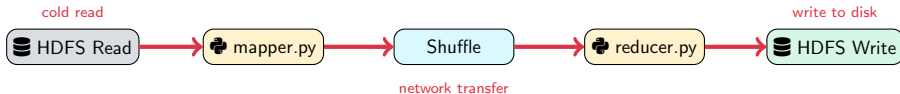
2 MapReduce Benchmark Results

Running the MapReduce Benchmark

```
# Step 1: Upload corpus to HDFS
python3 scripts/01_setup_data.py --student akoubaa

# Step 2: Run 3 MapReduce jobs
bash scripts/02_run_mapreduce.sh /user/akoubaa/lab07
```

Hadoop Streaming is used — Python mapper and reducer scripts run on worker nodes via YARN:



Repeated 3×

The entire pipeline above is executed **independently** for each of the 3 jobs. HDFS is read from scratch every time.

MapReduce: Actual Terminal Output (Job 1)

```
=====
SE446 Lab 07 -- MapReduce Benchmark
=====
Verifying HDFS input exists...
Input file: /user/akoubaa/lab07/corpus.txt (50.0 M)

=====
Running MapReduce -- Job 1: Full Word Count
(reading corpus from HDFS -- this is a cold disk read)
=====
...
2026-03-03 18:41:17 INFO mapreduce.Job: Running job: job_1771402826595_0087
2026-03-03 18:41:34 INFO mapreduce.Job: map 100% reduce 50%
2026-03-03 18:41:36 INFO mapreduce.Job: map 100% reduce 100%
2026-03-03 18:41:39 INFO mapreduce.Job: Job completed successfully
    Map input records=631,664
    Reduce input groups=456
    Reduce output records=456

[OK] Job 1 complete in 128s -- 456 distinct words found
MR_JOB1_SECONDS=128
```

MapReduce: Top 10 Most Frequent Words (Job 3)

Rank	Word	Count
1	in	81,165
2	cluster	81,086
3	will	80,905
4	reduce	80,855
5	you	80,819
6	so	80,741
7	a	80,735
8	hadoop	80,734
9	with	80,722
10	job	80,697

```
# Read Job 3 output from HDFS
hdfs dfs -cat \
  /user/akoubaa/lab07/mr_out_3/part-* \
  | sort -k2,2 -rn \
  | head -10
```

 Cost: 193 seconds

MapReduce Job 3 read the **full 50 MB corpus from HDFS**, re-mapped all 631,664 lines, shuffled, and sorted — just to produce these 10 lines.

Compare to Spark Action 3

Spark produced the **identical** top 10 in **3.30 s** by sorting the already-cached in-memory RDD.

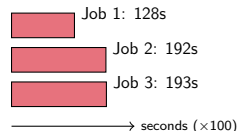
MapReduce: Results Summary

Job	Analysis	Time (s)	HDFS I/O
1	Full word count (456 distinct words)	128 s	Read 50 MB + Write
2	Words with count ≥ 100 (456 words)	192 s	Read 50 MB + Write
3	Top 10 most frequent words	193 s	Read 50 MB + Write
TOTAL		513 s	3× full HDFS reads



Observation

All 3 jobs take **roughly the same time**. Each pays the full disk cost — no learning from previous jobs.



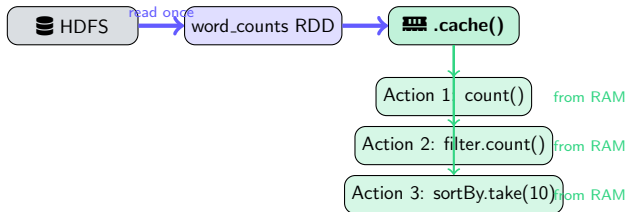
Outline

- 1 Lab Setup & Cluster
- 2 MapReduce Benchmark Results

Running the Spark Benchmark

```
spark-submit \  
  --master yarn \  
  --deploy-mode client \  
  --num-executors 2 \  
  --executor-memory 1g \  
  --executor-cores 1 \  
  scripts/03_run_spark.py /user/akoubaa/lab07/corpus.txt
```

The Spark execution flow for the **single** application:



Spark: Actual Terminal Output

```
Loading corpus from HDFS and building word-count RDD...
NOTE: data will be cached in executor RAM after the first action.
RDD is lazily defined; .cache() registered.
First action will trigger the actual HDFS read + computation.
```

```
=====
Action 1: Total distinct word count
=====
```

```
(This triggers the HDFS read + wordcount computation;
 the result is cached in executor RAM for the next actions)
Running: count() ...
[Done] Done in 21.30s
Result: 456 distinct words in the corpus
SPARK_ACTION1_SECONDS=21.30
```

```
=====
Action 2: Words with count >= 100 (reading from CACHE)
=====
```

```
Running: filter(count >= 100).count() ...
[Done] Done in 0.83s
SPARK_ACTION2_SECONDS=0.83
```

Spark: Top 10 Results

Rank	Word	Count
1	in	81,165
2	cluster	81,086
3	will	80,905
4	reduce	80,855
5	you	80,819
6	so	80,741
7	a	80,735
8	hadoop	80,734
9	with	80,722
10	job	80,697

About the Corpus

The synthetic corpus was generated from a fixed vocabulary of big-data keywords sampled uniformly. This explains why all counts are **nearly equal** ($\approx 80,800$) and why words like **hadoop**, **cluster**, **reduce** appear in the top 10.

Both Frameworks Agree

MapReduce and Spark produced **identical** word counts.

Total distinct words: **456**

Words with count ≥ 100 : **456** (all of them)

Spark: Results Summary

Action	Analysis	Time (s)	Data Source
1	Total word count	21.30 s	HDFS (cold) + cache to RAM
2	Words with count ≥ 100	0.83 s	 Executor RAM
3	Top 10 most frequent	3.30 s	 Executor RAM
TOTAL		25.43 s	1 HDFS read total

The Caching Effect

Action 1 pays the full I/O cost.

Actions 2 & 3 are **25×** and **6×** faster than Action 1 because they bypass HDFS entirely.



Outline

1 Lab Setup & Cluster

2 MapReduce Benchmark Results

Side-by-Side: The Numbers

Analysis	MapReduce	Spark	Speedup
Pass 1 / Action 1 — full word count	128 s	21.30 s	6.0×
Pass 2 / Action 2 — words ≥ 100	192 s	0.83 s	231×
Pass 3 / Action 3 — top 10	193 s	3.30 s	58.5×
TOTAL	513 s	25.43 s	20.2×



Speedup Formula

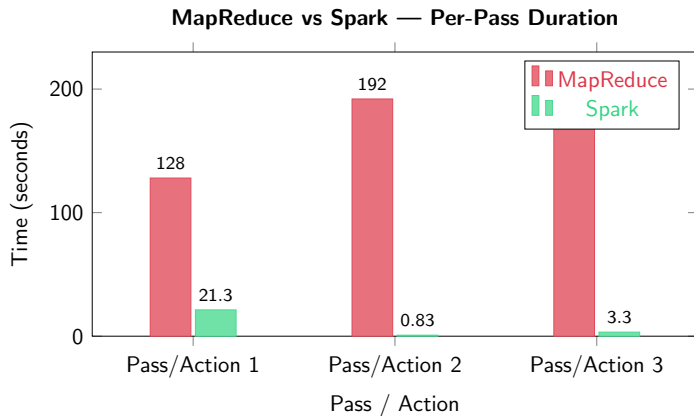
$$\text{Speedup} = \frac{T_{\text{MapReduce}}}{T_{\text{Spark}}} = \frac{513}{25.43} \approx \mathbf{20.2\times}$$



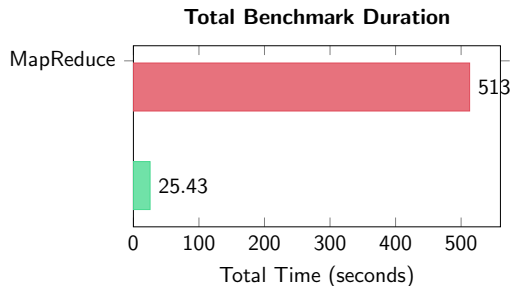
Headline Result

Spark completed the 3-pass analysis in **25 seconds** vs **513 seconds** for MapReduce. That is **8.5 minutes saved** on a **50 MB file**. Imagine 1 TB.

Visual Comparison: Per-Pass Timings



Visual Comparison: Total Time



Time Breakdown

MapReduce:

- Job 1: 128 s (25%)
- Job 2: 192 s (37%)
- Job 3: 193 s (38%)

Spark:

- Action 1: 21.30 s (84%)
- Action 2: 0.83 s (3.3%)
- Action 3: 3.30 s (13%)

Notice

84% of Spark's time is in Action 1 (cold read). That is the **only** time Spark touches disk.

Outline

1 Lab Setup & Cluster

2 MapReduce Benchmark Results

Pass 1 Comparison: Why is Spark Still 6× Faster?

MapReduce Job 1 — 128 s

- 1 YARN launches containers (JVM startup)
- 2 mapper.py reads 50 MB from HDFS
- 3 Map output spilled to **local disk**
- 4 Shuffle transfers data across network
- 5 reducer.py reads shuffle from disk
- 6 Output written to **HDFS**

✗ Intermediate results touch disk **twice**

Spark Action 1 — 21.30 s

- 1 YARN launches executors (JVM — *reused!*)
- 2 textFile reads 50 MB from HDFS
- 3 flatMap + map stay **in RAM pipeline**
- 4 shuffle for reduceByKey (once)
- 5 Result cached in executor RAM

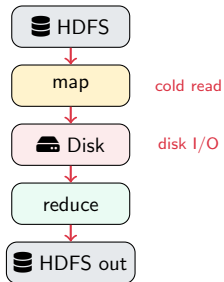
✓ Intermediate results never touch disk

💡 Key Insight

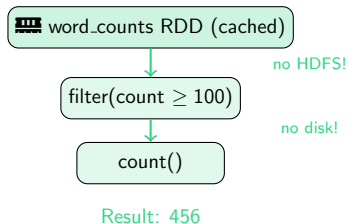
Even on the **first** pass, Spark wins because it pipelines transformations in RAM and avoids writing intermediate results to disk. MapReduce must serialize map output to disk before the shuffle.

Passes 2 & 3: Cache in Action

MapReduce — Job 2



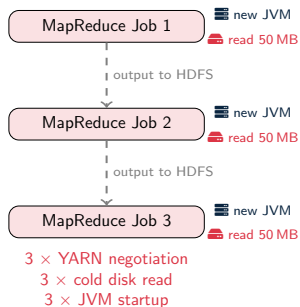
Spark — Action 2



Result

MapReduce Job 2: **192 s** (full HDFS read) Spark Action 2: **0.83 s** (pure RAM scan) Speedup: **231×**

Architecture: Why MapReduce Jobs Cost the Same Each Time



Every MapReduce job:

- Submits a new application to YARN
- Negotiates containers from ResourceManager
- Starts a fresh JVM on each worker
- Reads the **entire corpus from HDFS**
- Writes output back to HDFS
- JVMs shut down; all memory released

No Shared Memory

MapReduce has **no concept** of caching data between jobs. Every job starts from zero.

Outline

- 1 Lab Setup & Cluster
- 2 MapReduce Benchmark Results

Projection: What If We Run 10 Passes?

Linear scaling model:

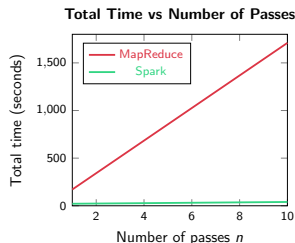
MapReduce:

$$T_n^{\text{MR}} = n \times \bar{t}_{\text{job}} \approx n \times 171 \text{ s}$$

Spark:

$$T_n^{\text{Spark}} = t_1 + (n - 1) \bar{t}_{\text{cached}}$$
$$\approx 21.3 + (n - 1) \times 2.07 \text{ s}$$

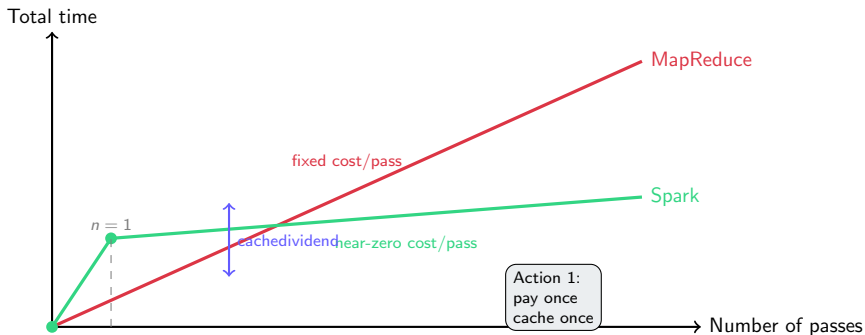
n	MR (s)	Spark (s)	Speedup
3	513	25	20×
5	855	30	~ 28×
10	1,710	40	~ 43×
100	17,100	227	~ 75×



Implication for Machine Learning

A gradient descent algorithm with 50 iterations on HdFS data would take MapReduce ~**2.4 hours** vs Spark ~**2 minutes** on this cluster.

The Bigger Picture: When Does Caching Pay Off?



💡 Rule of Thumb

If you access the **same dataset more than once** in a session (ML iterations, multiple aggregations, interactive exploration) $\Rightarrow$ **always use Spark with `.cache()`**

Outline

- 1 Lab Setup & Cluster
- 2 MapReduce Benchmark Results

Measured Conclusions

① Spark is faster even on the first pass ($6\times$).

Spark pipelines transformations in RAM and avoids intermediate disk writes. MapReduce serializes map output to local disk before every shuffle.

② Caching makes passes 2+ nearly free ($231\times$ speedup).

Once `.cache()` is triggered, Actions 2 and 3 are pure RAM scans. MapReduce re-reads 50 MB from HDFS for each job.

③ Total speedup compounds with number of iterations.

3 passes = $20\times$. 10 passes $\approx 43\times$. Any iterative algorithm (ML, graph, simulation) benefits enormously.

④ Both frameworks produce identical results.

456 distinct words, same top-10, same counts. The *what* is the same; the *how fast* is radically different.

⑤ MapReduce is not “wrong” — it is suited to different workloads.

Single-pass ETL on data that does not fit in RAM: MapReduce's fault tolerance and HDFS

Conceptual Summary: Architecture Dictates Performance

Property	MapReduce	Spark
Intermediate results	HDFS (disk)	Executor RAM
Data reuse between jobs	None — fresh read each time	.cache() — RAM served
JVM lifetime	Per job, then shutdown	Long-lived across all actions
YARN containers per analysis	$\sim 5/\text{job} \times 3 = \mathbf{15}$	3 (reused throughout)
Fault tolerance	HDFS replication	Lineage recomputation
Pass 1 time (50 MB)	128 s	21.30 s
Pass 2+ time (50 MB)	~ 190 s	< 1 s
Total (3 passes)	513 s	25.43 s

What's Next

Week 8A: Spark Performance Tuning

- spark-submit deep dive: partitioning, memory, parallelism
- .cache() vs .persist(MEMORY_AND_DISK) — storage levels
- Broadcast joins for small lookup tables
- Reading the Extended Spark Web UI (Stages, Tasks, Skew detection)

Week 8B: Spark MLlib

- Machine learning pipelines on the same cluster
- Each ML iteration reuses cached data — exactly what Lab 07 demonstrated

Lab 07 → MLlib Connection

The caching advantage you measured today ($231\times$) is the **same principle** that makes Spark MLlib practical: gradient descent takes 50+ iterations on the same dataset. Without caching, that would be 50 HDFS reads.

Remember

More passes on the same data $\Rightarrow$ greater Spark advantage